

Gebze Technical University
Department of Computer Engineering
CSE 241/501
Fall 2023
Midterm Exam Nov 22nd 2023

Student Code	Q1 (20)	Q2 (35)	Q3 (45)	Exam Format (-10)
230				

Read the instructions below carefully

- Do not write your name or your student number in any part of the exam materials including the answer sheets.
- Your individual student codes are already printed on your question and answer sheets. Keep this student code until the exam ends. You will use it to sign the exam attendance sheet.
- Answer each question on the designated page. Just put a single letter for each box. We will not (and cannot) grade any materials outside of the designated boxes.
- Hand over your cell phones, books, and notes to the TA before the exam starts. You will not use any electronic devices or books during the exam.
- This exam has 3 questions; it will be graded out of 100 points. You have 120 minutes. If you do not follow any of the rules above, you will lose 10 points.

Question 1

- Explain why **assert** is implemented as a macro instead of a global function?
- Why do we prefer the operator **new** over the function **malloc()** in C++? Explain your answer.

Question 2

Class **Point2D** represents a two dimensional point such as (3.5, 5.0). It has the following functions as well as any other functions that you think are necessary (setters, getters, etc.).

- A constructor that takes all parameters. All arguments should be defaulted.
- A function that returns the number of **Point2D** objects created so far.
- A function that returns the number of **Point2D** objects living at the time.
- Overloaded stream insertion operator (**cout << point**) will print **Point2D** details.
- Overloaded **==** and **!=** operators for comparing **Point2D**'s.
- Overloaded pre and post increment operators that increment the point components by 1.0. For example, (1.2, 3.5) becomes (2.2, 4.5) after increment.
- Overloaded unary **operator!** that returns a new **Point2D** object by switching the X and Y components of the **Point2D**.
- Overloaded binary **operator-** that returns the Euclidian distance between the given points as a double value.

Q2 Part a) Write the header file **Point2D.h** for the above class. Do not implement any functions in the header other than very short one-line functions.

Q2 Part b) Write the implementation file **Point2D.cpp** for the above class.

Question 3

Study the following driver code for a class named **GTUSet**.

```
#include <iostream>
#include "GTUSet.h"
using std::cout;

int main() {
    // Make set1 and insert numbers 1, 2, 3
    GTU::GTUSet set1;
    set1.insert(1);
    set1.insert(2);
    set1.insert(3);

    // Make set2 and insert numbers 3, 4, 5
    GTU::GTUSet set2;
    set2.insert(3);
    set2.insert(4);
    set2.insert(5);

    // Print the intersection of set1 and set2
    // Prints 3
    cout << (set1 ^ set2) << std::endl;

    // Remove 1 from set1
    set1.erase(1);

    // Remove 5 from set2
    set2.erase(5);

    // Print the union of set1 and set2
    // prints 2, 3, 4
    cout << (set1 + set2) << std::endl;

    // prints True
    cout << (set1 + set2).find(3) << std::endl;

    return 0;
}
```

Class **GTUSet** has three data members only: an **int *** to keep the integers and two **ints** to keep the capacity and the size of the set.

Q3 Part a) Write the complete class header. Do not implement any functions in the header.

Q3 Part b) Write the implementation file that includes all the necessary class member and global functions. Do not forget to check for errors. Remember, sets include only unique elements.

Write your answer for **Question 1-a** in the boxes below. Write one letter for each box.

The main reason is that we use assert for specific cases. If it was global, we could have called this function every single part of our code. Calling from everywhere increases the mistake rate and we could not understand where the specific problem happened.

Even if it was usable, I know from you that "we don't like global functions".

Write your answer for **Question 1-b** in the boxes below. Write one letter for each box.

As you said on lectures, C++ is improved language and we are not writing C anymore. New is easier than malloc thus we can implement the memory allocation more easily and we may prevent memory leaks.

New can be perform everything that malloc can and increases the readability of code.

New is newer, shorter, easier and it was specialized for C++. That is why we use it.

Write your answer for **Question 2-a** in the box below. Write one letter for each box.

```
#ifndef POINT2D-H
#define POINT2D-H

#include <iostream>

using namespace std;

class Point2D {
public:
    Point2D(double x=0.0, double y=0.03) {
        number = 1;
        cout << "Point2D constructor" << endl;
    }
    ~Point2D() {
        cout << "Point2D destructor" << endl;
    }
    int number = 1;
    double x, y;
    double get_x() const { return x; }
    double get_y() const { return y; }
    double set_x(double val) { x = val; }
    double set_y(double val) { y = val; }
    bool operator==(const Point2D& other) const {
        return (x == other.x && y == other.y);
    }
    bool operator!=(const Point2D& other) const {
        return !(*this == other);
    }
    Point2D operator++() {
        x++;
        y++;
        return *this;
    }
    Point2D operator++(int ignore) {
        x++;
        y++;
        return *this;
    }
    Point2D operator--() {
        x--;
        y--;
        return *this;
    }
    friend double operator-(const Point2D& p1, const Point2D& p2) {
        return p1.x - p2.x;
    }
private:
    double x;
    double y;
    static int count;
};

double operator-(int index) {
    return index;
}

#endif
```


Write your answer for **Question 2-b** in the box below. Write one letter for each box.

```
#include "Point2D.h"
#include <iostream>
#include <math.h>
using namespace std;

int Point2D::count = 0;

Point2D::Point2D(double x, double y): x(-x), y(-y) {
    count++;
}

ostream & operator<<(const Point2D & p) {
    cout<<p.x<<" "<<p.y<<endl;
    return cout;
}

bool Point2D::operator==(const Point2D & p) const {
    return ((x==p.x) && (y==p.y));
}

bool Point2D::operator!=(const Point2D & p) const {
    return ((x!=p.x) || (y!=p.y));
}

Point2D & Point2D::operator++() {
    x++;
    y++;
    return *this;
}

Point2D Point2D::operator++(int ignore) {
    double tempX = x;
    double tempY = y;
    x++;
    y++;
    return Point2D(tempX, -tempY);
}

Point2D Point2D::operator!() {
    return Point2D(y, x); // constructor
}

double operator-(const Point2D & p, double x1, double y1) {
    double dist = 0.0;
    dist = sqrt(pow(abs(x-x1), 2) + pow(abs(y-y1), 2));
    return dist;
}

double & Point2D::operator[](int index) {
    if (index == 0) return x;
    if (index == 1) return y;
}
```


Write your answer for **Question 3-a** in the box below. Write one letter for each box.

```
#ifndef GTUSET_H
#define GTUSET_H
#include <iostream>
namespace
{
    class GTUSet
    {
    public:
        GTUSet();
        void insert(int num);
        void erase(int num);
        const operator+(const GTUSet& a) const;
        bool find(int num) const;
        friend std::ostream& operator<<(std::ostream& out, const GTUSet& s);
        void increaseCap();
        GTUSet(const GTUSet& obj); // Not used
        GTUSet operator+(const GTUSet& g) const;
        ~GTUSet() { delete[] do+o; do+o=NULLptr; }
        // No place for destructor, I'm sorry)

    private:
        int *do+o;
        int capacity;
        int size;
    };
}

#endif
```


Write your answer for **Question 3-b** in the box below. Write one letter for each box.

```
#include "GTUSet.h"
#include <iostream>

namespace GTUSet {

    GTUSet * GTUSet::GTUSet(int size=10, capacity=10, do+o(nullptr)
    r){
        do+o=new int[capacity];
    }

    void GTUSet::incrSize-cap() {
        capacity *= 2;
        int * temp=new int[capacity];
        for(int i=0; i<size; i++){
            temp[i]=do+o[i];
        }
        delete[] do+o;
        do+o=temp;
    }

    void GTUSet::insert(int num){
        if(size+1>capacity) incrSize-cap();
        do+o[size++]=num;
    }

    void GTUSet::erase(int num){
        int i=0, k=0;
        while(k<size){
            if(num==do+o[k]){
                flag=1;
                k++;
            }
            if(flag==1) return;
            do+o[k]=temp[k];
            GTUSet temp;
            for(int i=0; i<size; i++){
                if(num!=do+o[i]) temp.insert(do+o[i]);
            }
            delete[] do+o;
            size=temp.size;
            do+o=temp.do+o;
        }
    }
}
```


If you need extra space, you should write your answer for **Any Question** in the box below.

```
static const char *opero = "0123456789+-*/%<>";
static const char *opero = "0123456789+-*/%<>";
```

```
for (int i = 0; i < size; i++) {
    opero[i] = "0123456789+-*/%<>";
}
```

```
return opero;
}
```

```
const char *opero = "0123456789+-*/%<>";
const char *opero = "0123456789+-*/%<>";
```

```
union {
    int flag;
```

```
for (int i = 0; i < size; i++) {
```

```
    union.insert(i, "0123456789+-*/%<>");
}
```

```
for (int j = 0; j < 9; j++) {
    flag = 0;
```

```
    for (int k = 0; k < inter.size(); k++) {
```

```
        if (union[i][k] == "0123456789+-*/%<>")
            flag = 1; break;
    }
```

```
    if (!flag) {
```

```
        inter.insert(i, "0123456789+-*/%<>");
    }
```

```
return union;
}
```

```
const char *opero = "0123456789+-*/%<>";
const char *opero = "0123456789+-*/%<>";
```

```
union {
    int inter;
```

```
for (int i = 0; i < size; i++) {
```

```
    inter.insert(i, "0123456789+-*/%<>");
}
```

```
for (int j = 0; j < 9; j++) {
```

```
    for (int k = 0; k < inter.size(); k++) {
```

```
        if (inter[i][k] != "0123456789+-*/%<>") {
```

```
            inter.erase(inter[i][k]);
        }
```

```
    }
    return inter;
}
```

```
bool find(int num) {
```

```
    for (int i = 0; i < size; i++) {
```

```
        if (opero[i] == num) return true;
    }
```

```
return false;
}
```

3/11 No more space